# CampusKart: An AI-Enabled Database System for a Peer-to-Peer Student Marketplace

[Author Name 1]
*[Dept. of Computer Engineering]*
*[College / University Name]*
[City], India
[email address]

[Author Name 2]
*[Dept. of Computer Engineering]*
*[College / University Name]*
[City], India
[email address]

[Author Name 3]
*[Dept. of Computer Engineering]*
*[College / University Name]*
[City], India
[email address]

***Abstract*—Students in colleges regularly buy expensive textbooks, calculators, drawing instruments and hostel items that their seniors sell or throw away a year later. This paper presents CampusKart, a campus-only second-hand marketplace built around a normalized relational database and three lightweight machine learning modules. The database, implemented in SQLite with nine tables in third normal form, enforces business rules through six triggers, four views, foreign keys and CHECK constraints, and runs every purchase as a single atomic transaction so that an item can never be sold twice. Using the stored search history, product views and wishlists, a content-based recommender based on TF-IDF and cosine similarity produces personalised and explainable suggestions. A linear regression model trained on existing listings suggests a fair resale price, and a Multinomial Naive Bayes classifier combined with simple rules flags spam or fraudulent listings for administrator review. On the demonstration dataset the recommender achieved a precision at five of 1.00 for all four test profiles, the price model reached a mean absolute percentage error of 7.2 percent under five-fold cross validation, and all 56 genuine listings passed the spam filter while both spam listings were caught. Every AI response completes in under 10 ms.**

***Keywords*—database management system, relational schema, triggers, transactions, recommender system, TF-IDF, Naive Bayes, linear regression**

## I. Introduction

Every academic year, students buy new textbooks, scientific calculators, lab coats, drafters and hostel essentials, while graduating seniors discard the same items or sell them informally through messaging groups. Such groups have no search, no price reference, no record of who sold what, and no protection against fraudulent posts. General marketplaces such as OLX or Facebook Marketplace are open to the public, which makes meeting strangers and paying in advance risky for students.

A database-backed campus marketplace solves most of these problems. Listings become structured records, every purchase becomes a transaction, and the history of searches and views becomes data that can help each student find what they need. The aim of this project is to design such a system on sound database principles first, and then add a small number of explainable AI features that run directly on the stored data.

The main contributions of this work are:

- A normalized relational schema (3NF and BCNF) of nine tables derived from an ER model, with integrity enforced by keys, CHECK constraints, six triggers and four views.
- A transaction design that prevents an item from being sold twice, even when two buyers act at the same moment.
- Three AI modules that use the database as their training data: a TF-IDF recommender driven by search history, a regression-based price suggester and a Naive Bayes spam detector.
- A complete web application with student and administrator roles, including a read-only SQL explorer that demonstrates the database live.

Section II reviews related work and Section III lists the requirements. Section IV presents the system and database design, Section V the AI modules, Section VI the implementation and Section VII the results. Section VIII concludes the paper.

## II. Literature Survey

### *A. Relational Databases and ER Modelling*

Codd's relational model [3] and Chen's entity-relationship model [4] remain the foundation of database design, and standard textbooks [1], [2] describe how an ER model is mapped to relations and normalized. Triggers and views [1] allow business rules to be enforced inside the DBMS instead of in application code, and ACID transactions [12] keep data consistent under concurrent access.

### *B. Recommender Systems*

Collaborative filtering, popularised by Amazon's item-to-item approach [13], needs many users with overlapping purchases, which a single campus does not have at launch. Content-based filtering [6] instead compares item descriptions with a profile built from the user's own behaviour, usually with TF-IDF weighting [5] and cosine similarity. Users without any history form the well-known cold-start problem [9], commonly handled with a popularity-based fallback.

### *C. Spam Detection and Price Estimation*

Naive Bayes classifiers have been used for junk e-mail filtering since [7], and the multinomial variant performs well on short texts [14]. Linear regression is the standard, interpretable baseline for price estimation. The scikit-learn library [8] provides tested implementations of all three techniques.

### D. Existing Platforms

Table I compares CampusKart with the platforms students use today. None of the existing options combines a campus-only audience with structured data, price guidance and fraud screening.

TABLE I.
COMPARISON WITH EXISTING PLATFORMS

| Feature | OLX / FB | Chat groups | CampusKart |
|---|---|---|---|
| Restricted to campus | No | Yes | Yes |
| Search and filters | Yes | No | Yes |
| Personal recommendations | Opaque | No | Explained |
| Fair price suggestion | No | No | Yes |
| Spam / fraud screening | Partial | No | AI + admin |
| Seller ratings | Partial | No | Yes |
| Double-sale prevention | No | No | Trigger |

## III. REQUIREMENT ANALYSIS

### A. Functional Requirements

- Registration and login with two roles: student and administrator.
- Listing an item with category, condition, MRP, age and price.
- Search with filters (category, condition, maximum price) and sorting.
- Wishlist, purchase, cancellation, receipt confirmation and seller rating.
- Notifications for sales, cancellations, wishlist items and moderation.
- AI recommendations, price suggestion and spam screening.
- Admin dashboard, listing moderation, user blocking and SQL explorer.

### B. Non-Functional Requirements

The system must never sell an item twice (consistency), store only hashed passwords and use parameterised queries (security), work on phones and laptops (usability), answer every request in under 100 ms (performance), and run on any laptop without installing a database server (portability).

### C. Hardware and Software Requirements

Any PC with 4 GB RAM and a modern browser is sufficient. The software stack is Python 3.12, Flask 3.1, SQLite 3, scikit-learn 1.8 and NumPy, with HTML5, CSS3 and JavaScript on the client.

### D. Use Case Model

Fig. 2 shows the two actors. A student registers, searches, receives recommendations, sells with a price hint, buys, cancels, rates sellers, manages a wishlist and reads notifications. An administrator reviews AI-flagged listings, blocks users, views reports and runs read-only SQL queries.

## IV. SYSTEM DESIGN

### A. System Architecture

CampusKart follows a three-tier architecture (Fig. 1). The presentation tier is a server-rendered web interface. The application tier, written in Flask, handles routing, sessions, authentication and business logic, and calls the three AI modules. The data tier is a single SQLite database that also serves as the training data for the AI models, so no separate data pipeline is needed.

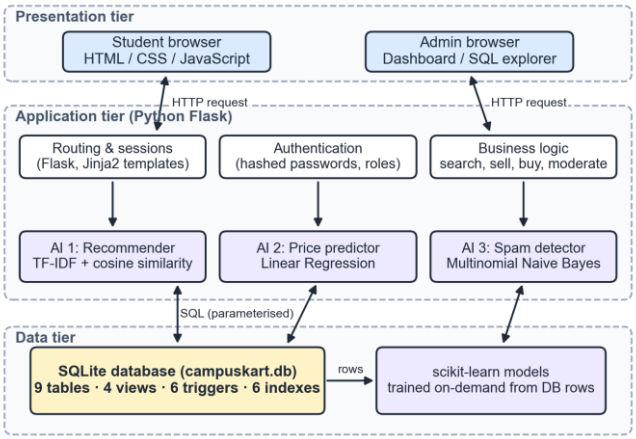



Fig. 1. Three-tier architecture of CampusKart.

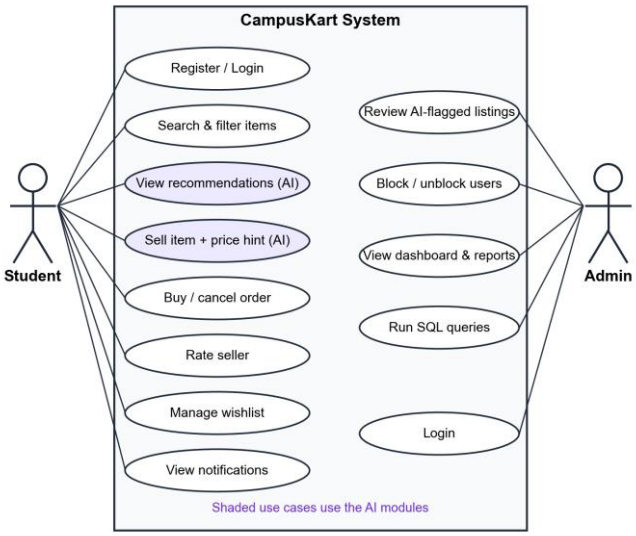



Fig. 2. Use case diagram.

### B. Data Flow Diagrams

The context diagram (Fig. 3) treats the whole system as one process exchanging data with the two external entities. The level-1 DFD (Fig. 4) decomposes it into six processes: authenticate user, search and browse, manage listings, process orders, generate recommendations, and moderate and report. These processes use five data stores: Users, Products, Orders and Reviews, Search History and Views, and Notifications. The flows marked as triggers are produced by the DBMS itself, not by the application.

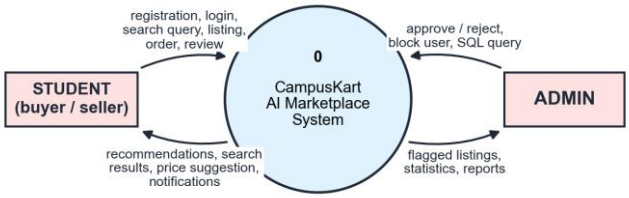



Fig. 3. Level-0 (context) data flow diagram.

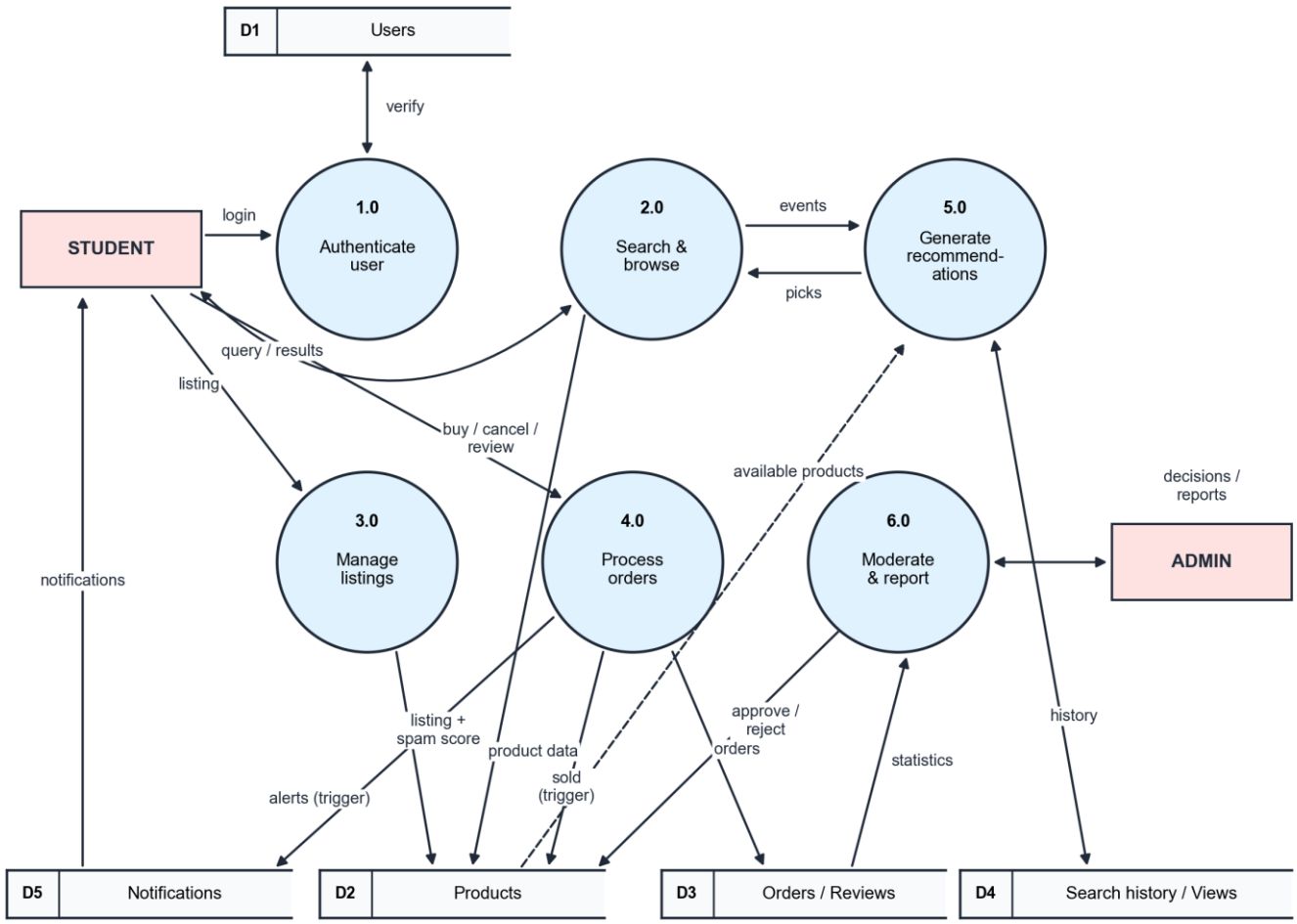



Fig. 4. Level-1 data flow diagram.

### *C. Entity-Relationship Model*

The ER diagram in Fig. 5 uses Chen notation and contains seven entity sets: USER, PRODUCT, CATEGORY, ORDER, REVIEW, SEARCH_HISTORY and NOTIFICATION. The relationships and their cardinalities are:

- SELLS (USER 1 : N PRODUCT) and BELONGS_TO (PRODUCT N : 1 CATEGORY), both with total participation of PRODUCT.
- PLACES (USER 1 : N ORDER), CONTAINS (ORDER N : 1 PRODUCT) and HAS (ORDER 1 : 1 REVIEW).
- WISHLISTS and VIEWS (USER M : N PRODUCT), with descriptive attributes added_at and viewed_at.
- PERFORMS (USER 1 : N SEARCH_HISTORY) and RECEIVES (USER 1 : N NOTIFICATION).

A product can appear in several orders over time because a cancelled order puts it back on sale, which is why CONTAINS is N : 1 and not 1 : 1.

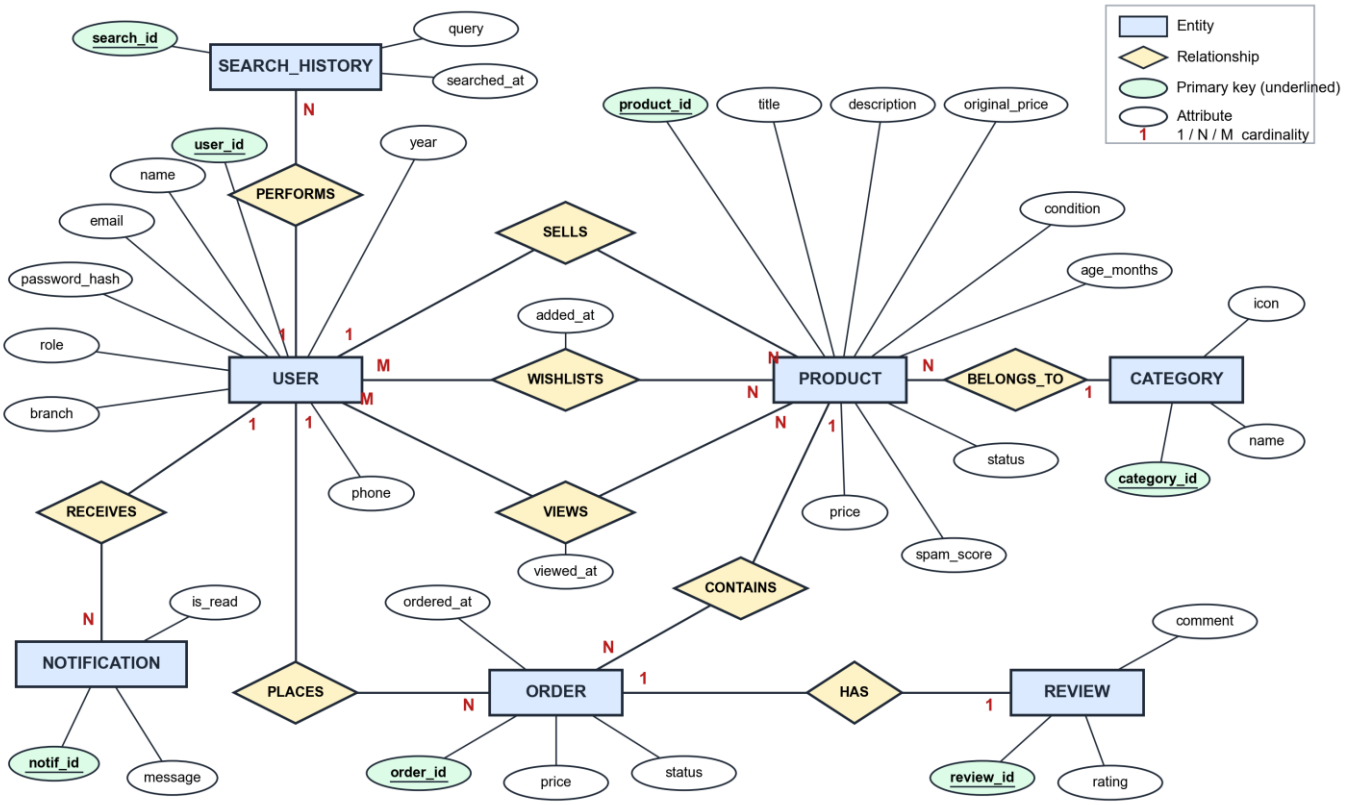



Fig. 5. Entity-relationship diagram of CampusKart (Chen notation).

*D. Relational Schema*

The ER model was mapped to relations with the standard rules [2]. Each strong entity set became a table with a surrogate integer primary key. Each 1 : N relationship became a foreign key on the N side, and the 1 : 1 HAS relationship became a foreign key with a UNIQUE constraint in reviews. The M : N WISHLISTS relationship became the table wishlist with the composite primary key (user_id, product_id). VIEWS uses a surrogate key view_id instead, because the recommender needs every repeated view with its timestamp. The resulting schema is shown in Fig. 6 and summarised in Table II.

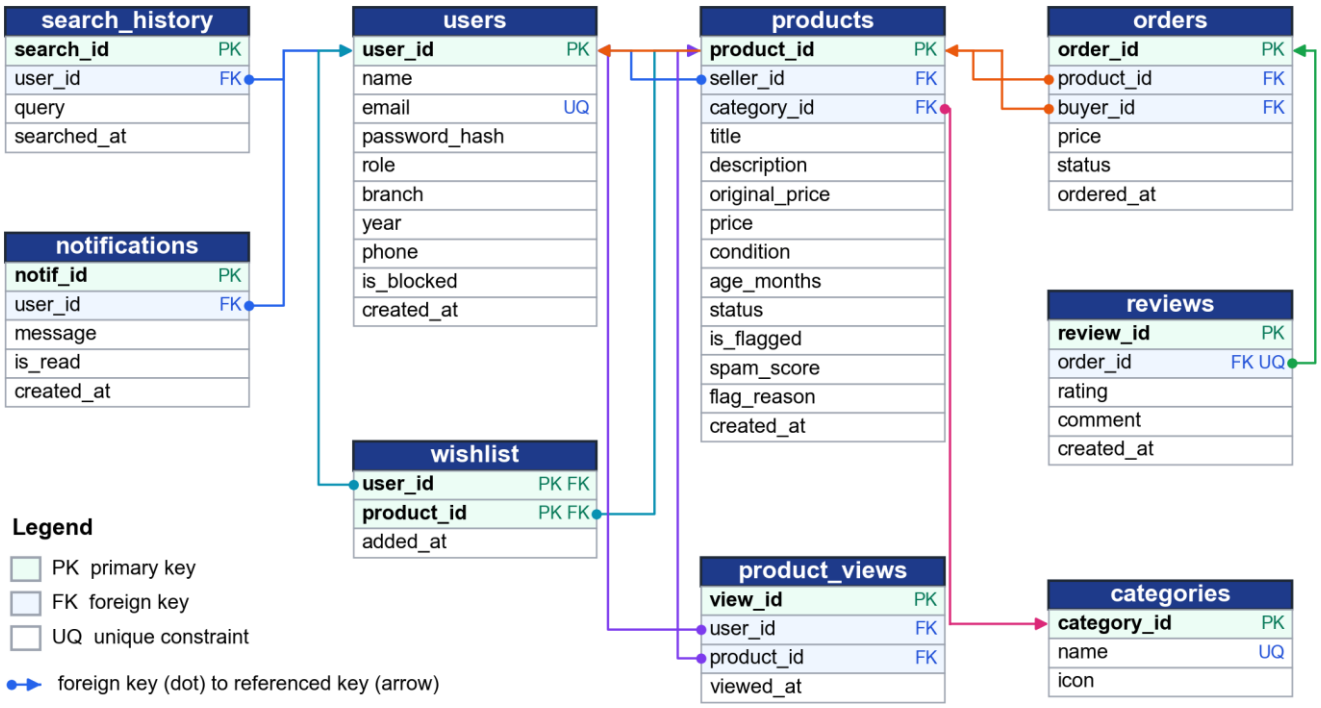



Fig. 6. Relational schema with primary keys, foreign keys and unique constraints.

TABLE II.
TABLES IN THE DATABASE

| Table | Primary key | Foreign keys | Rows |
|---|---|---|---|
| users | user_id | – | 12 |
| categories | category_id | – | 9 |
| products | product_id | seller_id, category_id | 58 |
| orders | order_id | product_id, buyer_id | 7 |
| reviews | review_id | order_id (unique) | 5 |
| wishlist | (user_id, product_id) | user_id, product_id | 4 |
| search_history | search_id | user_id | 21 |
| product_views | view_id | user_id, product_id | 70 |
| notifications | notif_id | user_id | 22 |

The demonstration database holds 208 rows and occupies only 108 KB, which keeps it small enough to inspect during a viva while still exercising every relationship.

### E. Normalization

**First normal form:** every attribute is atomic. A student's wishlist, for example, is stored as rows of a separate table and not as a list inside users.

**Second normal form:** the only composite key is (user_id, product_id) in wishlist, and its single non-key attribute added_at depends on the whole key. All other tables have single-attribute keys, so partial dependencies cannot occur.

**Third normal form:** transitive dependencies were removed during design. Storing seller_name in products would create product_id → seller_id → seller_name, so only seller_id is kept. Likewise, orders does not store seller_id, because order_id → product_id → seller_id. Derived values such as a seller's average rating are computed by views instead of being stored, which avoids update anomalies.

In every table each determinant is a candidate key (for example, email is a candidate key of users), so the schema also satisfies Boyce-Codd normal form.

### F. Constraints, Triggers, Views and Indexes

Domain rules are declared as CHECK constraints: price > 0, rating between 1 and 5, year between 1 and 4, and enumerated values for role, condition and status. Referential integrity uses foreign keys with ON DELETE CASCADE where child rows have no meaning on their own. Table III lists the six triggers that move business logic into the DBMS.

TABLE III.
DATABASE TRIGGERS

| Trigger | Event | Action |
|---|---|---|
| trg_order_validate | BEFORE INSERT orders | Abort if item not available or buyer is seller |
| trg_order_placed | AFTER INSERT orders | Mark product sold, notify seller |
| trg_order_cancelled | AFTER UPDATE orders | Re-list product, notify seller |
| trg_wishlist_sold_alert | AFTER UPDATE products | Notify users who wishlisted it |
| trg_flagged_listing | AFTER INSERT products | Alert admins about AI-flagged listing |

| Trigger | Event | Action |
|---|---|---|
| trg_listing_moderated | AFTER UPDATE products | Tell seller approved / rejected |

The validation trigger is the key safeguard against double selling:

```
CREATE TRIGGER trg_order_validate
BEFORE INSERT ON orders
BEGIN
  SELECT RAISE(ABORT,
         'This product is no longer available')
  WHERE (SELECT status FROM products
         WHERE product id = NEW.product id)
        <> 'available';
  SELECT RAISE(ABORT,
         'You cannot buy your own product')
  WHERE (SELECT seller_id FROM products
         WHERE product id = NEW.product id)
        = NEW.buyer_id;
END;
```

Four views provide reusable read-only queries: v_product_details (a four-table join used by every listing page), v_seller_ratings (AVG with GROUP BY), v_category_sales (LEFT JOIN with SUM) and v_admin_stats (scalar subqueries for the dashboard). Six indexes, such as products(status, category_id) and search_history(user_id, searched_at), support the most frequent queries.

### *G. Transaction Management*

Every write runs inside a transaction: the application opens it with BEGIN and ends with COMMIT on success or ROLLBACK on any error. Fig. 7 shows a purchase. Inserting the order fires the validation trigger, then the after-insert trigger marks the product sold and creates notifications. All of these steps commit together or not at all (atomicity). SQLite serialises writers with a database-level lock, so if two students press Buy at the same moment the second insert runs after the first, sees the status sold, and is aborted by the trigger (isolation). The rollback journal provides durability.

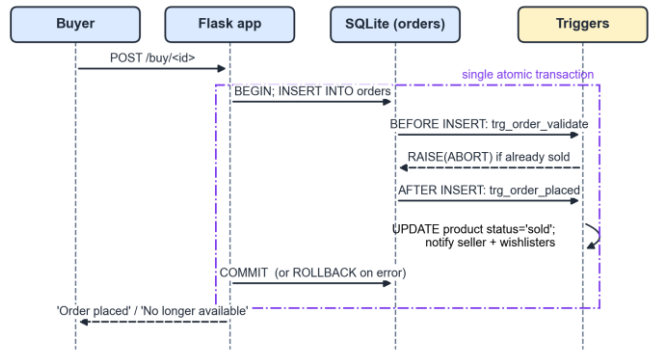



Fig. 7. Sequence of a purchase: one atomic transaction with triggers.

## V. AI Modules

### *A. Recommendations From Search History*

Each available product is converted into a TF-IDF vector built from its title (weighted twice), category and description. Character n-grams of length 3 to 5 are used instead of whole words, so partial or misspelled searches such as “arduno” still match “Arduino”. With sublinear term frequency and smoothed inverse document frequency, the weight of n-gram t in product d is

$$w(t, d) = (1 + \ln tf(t, d)) \cdot [\ln((1 + N) / (1 + df(t))) + 1] \quad (1)$$

where N is the number of products and df(t) the number of products containing t. Similarity between two vectors is the cosine

$$cos(a, b) = (a \cdot b) / (\|a\| \, \|b\|) \quad (2)$$

The user profile consists of signals read from the database: the last ten distinct searches, the last ten viewed products and the wishlist. A signal of rank r (0 for the newest) receives the weight $\beta \cdot 0.9^r$, where β is 1.0 for searches and wishlist items and 0.8 for views. The score of product p is the strongest weighted match

$$score(p) = \max_i w_i \cdot cos(s_i, p) \quad (3)$$

and matches with a cosine below 0.15 are ignored. The signal that produced the maximum becomes the explanation shown on the card, for example “Because you searched ‘python’”. Products already in the wishlist or sold by the user are excluded, and the top eight are displayed. When a user has no history (the cold-start case), the system shows the items viewed most in the last 30 days instead. The complete pipeline is shown in Fig. 8. The same TF-IDF index also powers the “Similar items” panel and a typo-tolerant “Did you mean” search.

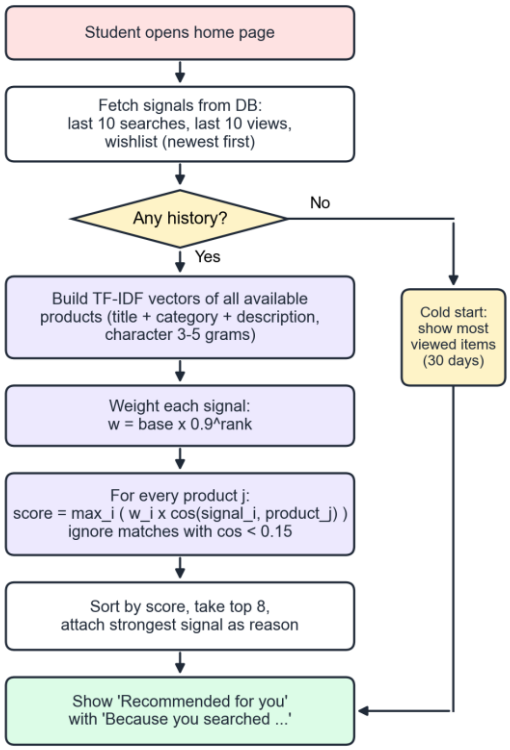



Fig. 8. Recommendation pipeline with cold-start fallback.

### *B. Fair Price Prediction*

Prices range from about Rs. 100 to Rs. 52,000, so instead of predicting the price directly the model predicts the resale ratio r = price / MRP, which lies on the same scale for every item.

The features are a one-hot encoding of the condition (New, Like New, Good, Fair), the age in years capped at five, and a one-hot encoding of the nine categories. An ordinary least-squares linear regression is trained on all approved listings each time a prediction is requested:

$$\hat{r} = \theta_0 + \Sigma_k \theta_k x_k, \quad price = clip(\hat{r}, 0.10, 0.95) \cdot MRP \qquad (4)$$

The suggestion is rounded and shown with a ±10% range. On the demonstration data the model learned that Like New items keep about 8 percentage points more of their MRP than Good items, Fair items about 8 points less, and every year of age costs about 3 points. The same model labels each product page for buyers as Great deal (at least 10% below the estimate), Fair price or Overpriced (more than 12% above).

### C. Spam and Fraud Detection

A Multinomial Naive Bayes classifier with TF-IDF word and bigram features is trained on 38 labelled genuine and spam listings. For a listing d it estimates

$$P(spam \mid d) \propto P(spam) \cdot \Pi_t P(t \mid spam) \qquad (5)$$

Three explainable rules complement the model: a phone number, link or off-platform payment word (UPI, GPay, OTP), a price above the MRP, and a price below 5% of the MRP. Each rule raises the score to at least 0.55 to 0.6, and a listing whose final score reaches 0.5 is stored with status pending. The insert fires trg_flagged_listing, which notifies every administrator, and the admin's decision fires trg_listing_moderated, which informs the seller. The sell page also calls the detector while the student types, so problems are visible before the listing is published.

## VI. Implementation

### A. Technology and Structure

The application consists of schema.sql (tables, indexes, triggers and views), seed.py (demonstration data), app.py (about 40 Flask routes), the ai package (recommender.py, price_predictor.py and spam_detector.py), 18 Jinja2 templates and one stylesheet with no external dependencies. SQL is written by hand rather than through an ORM so that every query is visible. A typical query, used for the profile page, is:

```
SELECT p.seller_id,
       ROUND(AVG(r.rating), 1) AS avg_rating,
       COUNT(r.review_id)      AS review_count
FROM reviews r
JOIN orders   o ON o.order id   = r.order id
JOIN products p ON p.product_id = o.product_id
GROUP BY p.seller_id;
```

### B. Security

Passwords are stored as salted PBKDF2 hashes. Every query uses parameter binding, which prevents SQL injection. Role checks protect all admin routes, blocked users are logged out on their next request, and the SQL explorer accepts only SELECT or WITH statements and runs them over a separate read-only connection (mode=ro), so even a crafted query cannot modify data. Users can view and clear their own search history, which controls what the recommender knows about them.

### C. Demonstration Data

The seed script creates an administrator and eleven students. Four demonstration students have distinct search histories: computer science (DBMS, Python, laptop), mechanical (engineering drawing, drafter, calculator), electronics (Arduino, multimeter) and hostel items (cycle, mattress, kettle). A fifth, new student has no history, which demonstrates the cold-start case. The database contains 58 products in nine categories, seven orders including one cancellation, five reviews and two spam listings that wait for moderation.

## VII. Results and Discussion

### A. Functional Testing

An automated test script drives the application through every page and flow. Table IV lists the most important cases, all of which pass.

TABLE IV.
Selected Test Cases

| Test case | Expected result | Status |
|---|---|---|
| Login with wrong password | Error message, no session | Pass |
| Blocked user logs in | Access denied | Pass |
| Buy an available item | Order placed, item sold, seller notified | Pass |
| Second buyer, same item | Aborted by trigger | Pass |
| Cancel an order | Item listed again | Pass |
| Review same order twice | Rejected by UNIQUE | Pass |
| Post a spam listing | Pending, admins notified | Pass |
| Admin approves listing | Live, seller notified | Pass |
| DELETE in SQL explorer | Rejected (read-only) | Pass |
| Search for "arduno" | AI suggests Arduino items | Pass |

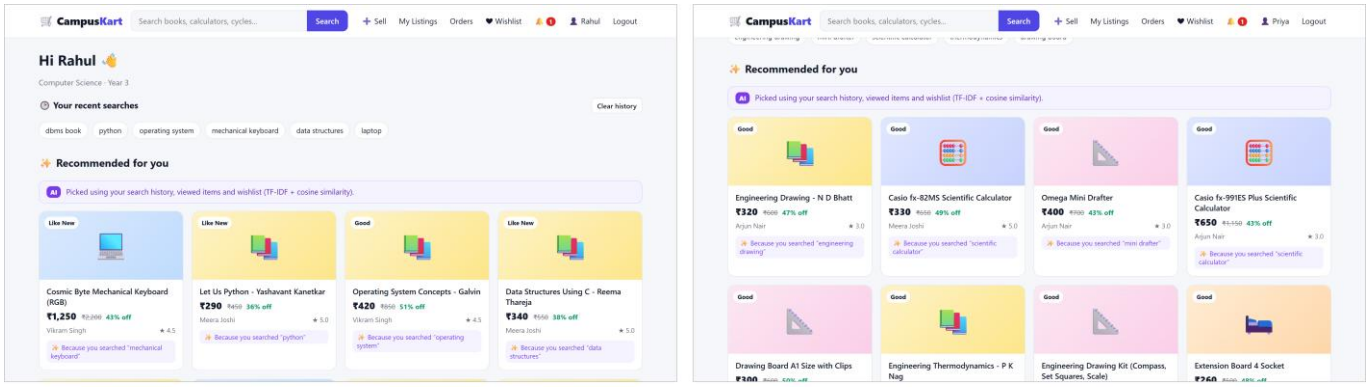
### B. Evaluation of the AI Modules

Table V summarises the measured results. Recommendation quality was measured as precision at five: the share of the top five suggestions that belong to the categories each demonstration profile is interested in. The price model was evaluated with five-fold cross validation on 56 listings, and the Naive Bayes model with leave-one-out validation on its 38 training examples. Response times are averages on a standard laptop.

TABLE V.
Evaluation of AI Modules

| Module | Metric | Result |
|---|---|---|
| Recommender | Precision@5 (4 profiles) | 1.00 |
| Recommender | Average response time | 7.9 ms |
| Price model | MAPE (5-fold CV) | 7.2% |
| Price model | $R^2$ on resale ratio | 0.55 |
| Price model | Average response time | 0.7 ms |
| Spam detector | Accuracy / F1 (LOO) | 1.00 / 1.00 |
| Spam detector | False alarms on 56 genuine listings | 0 |

These scores come from a small, clean demonstration dataset and should not be read as general accuracy; a real deployment would need more listings and labelled spam to confirm them. The character n-gram matching occasionally links unrelated words (for example “breadboard” and “drawing board”), and the 0.15 similarity threshold removes most of these weak matches.




(a) Rahul (CSE): searched dbms book, python, laptop

(b) Priya (Mechanical): searched engineering drawing, drafter

Fig. 9. Home pages of two students with different search histories: the same database gives each a different, explained recommendation list.

## C. User Interface

Fig. 9 shows the home pages of two demonstration students. With the same database, the mechanical student sees drawing instruments and calculators, while the computer science student sees programming books and a keyboard, each with the search that caused it. Fig. 10 shows the sell page: the AI suggests Rs. 690 for a calculator with an MRP of Rs. 1,200, and the spam checker reacts to the words “whatsapp me” before the listing is published. Fig. 11 shows the price badge on a product page, Fig. 12 the typo-tolerant search, Fig. 13 the moderation queue with the reasons for each flag, and Fig. 14 the SQL explorer running a join with GROUP BY.

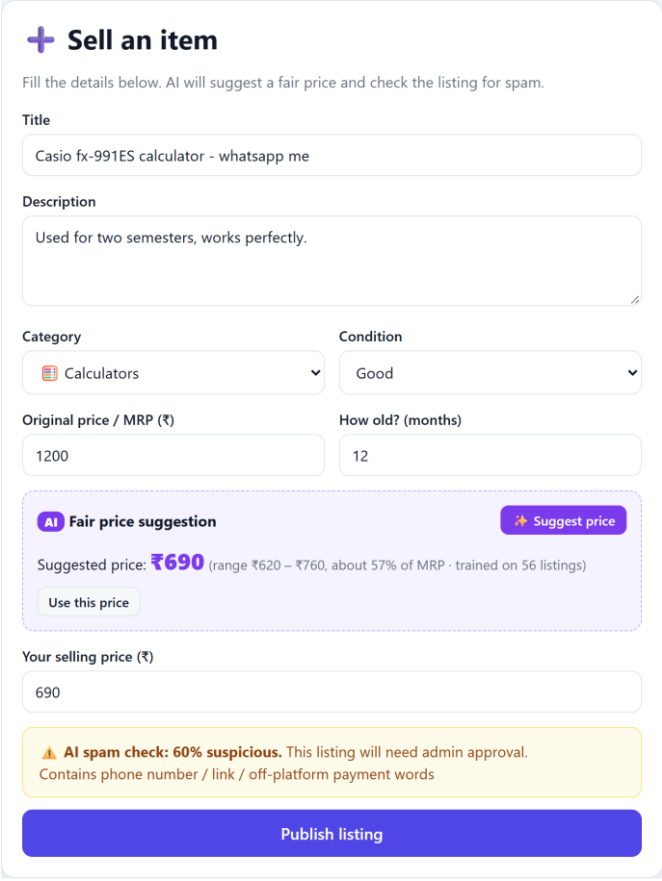



Fig. 10. Sell page with AI price suggestion and live spam check.

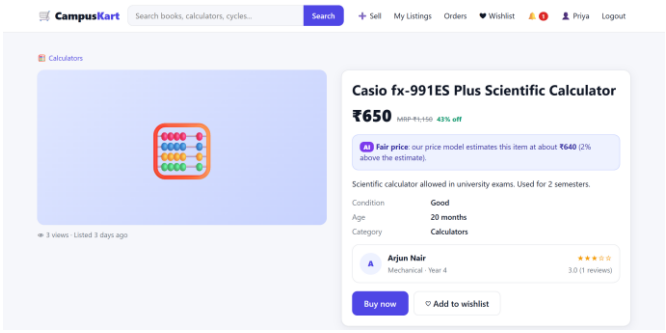



Fig. 11. Product page with the AI fair-price badge.

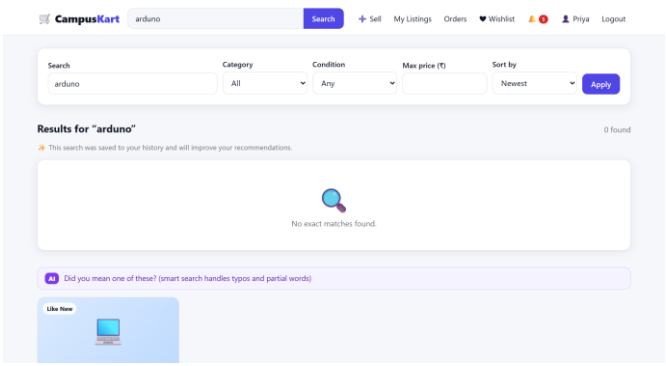



Fig. 12. No exact match for "arduno": the AI suggests close matches.

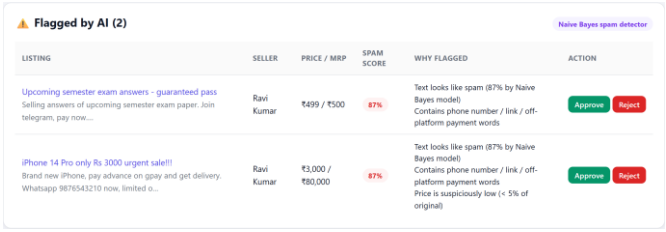



Fig. 13. Admin queue of listings flagged by the spam detector.

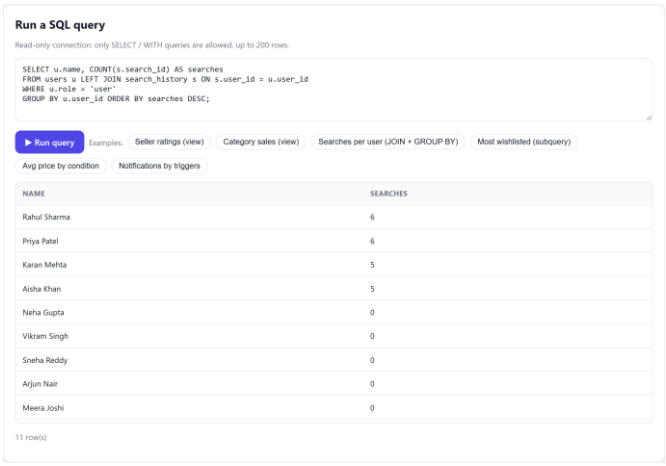



Fig. 14. Read-only SQL explorer in the admin panel.

## VIII. Conclusion and Future Scope

CampusKart shows that a carefully normalized database can serve both as the core of a transactional application and as the training data for useful AI features. Triggers, views and transactions keep the data consistent without extra application code, while TF-IDF recommendations, regression-based pricing and Naive Bayes screening make the marketplace easier and safer to use. All three AI modules are simple enough to explain step by step, and each of their outputs carries a reason that the user can see.

Future work includes migrating to MySQL or PostgreSQL for multi-campus use, adding collaborative filtering once enough purchase history exists, uploading product photographs with image-based duplicate detection, in-app chat between buyer and seller, and retraining the spam model on listings rejected by administrators.

## Acknowledgment

The authors thank Guide Name, Department, College Name, for guidance and support throughout this project.

## References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson, 2016.

[3] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] P. P. Chen, "The entity-relationship model—toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, 1976.

[5] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[6] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston: Springer, 2011, pp. 73–105.

[7] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian approach to filtering junk e-mail," in *Proc. AAAI Workshop on Learning for Text Categorization*, 1998, pp. 55–62.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[9] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in *Proc. 25th Annu. Int. ACM SIGIR Conf.*, 2002, pp. 253–260.

[10] SQLite Consortium, "SQLite documentation," 2024. [Online]. Available: https://www.sqlite.org/docs.html

[11] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA: O'Reilly, 2018.

[12] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann, 1993.

[13] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Comput.*, vol. 7, no. 1, pp. 76–80, 2003.

[14] V. Metsis, I. Androutsopoulos, and G. Paliouras, "Spam filtering with naive Bayes—Which naive Bayes?" in *Proc. 3rd Conf. Email and Anti-Spam (CEAS)*, 2006.